AI INTERVIEW COACH

System Wiring & Tools/Platform Guide

This guide shows exactly how every component of the app is wired together, and gives you concrete, real tool/platform names for every concept so you're never left wondering "okay, but what do I actually type into Android Studio?"

1. System Wiring Diagram

This is the actual technical architecture — which real tool handles which job, and how your Android app talks to each one. Every arrow is a real connection you'll write code for.

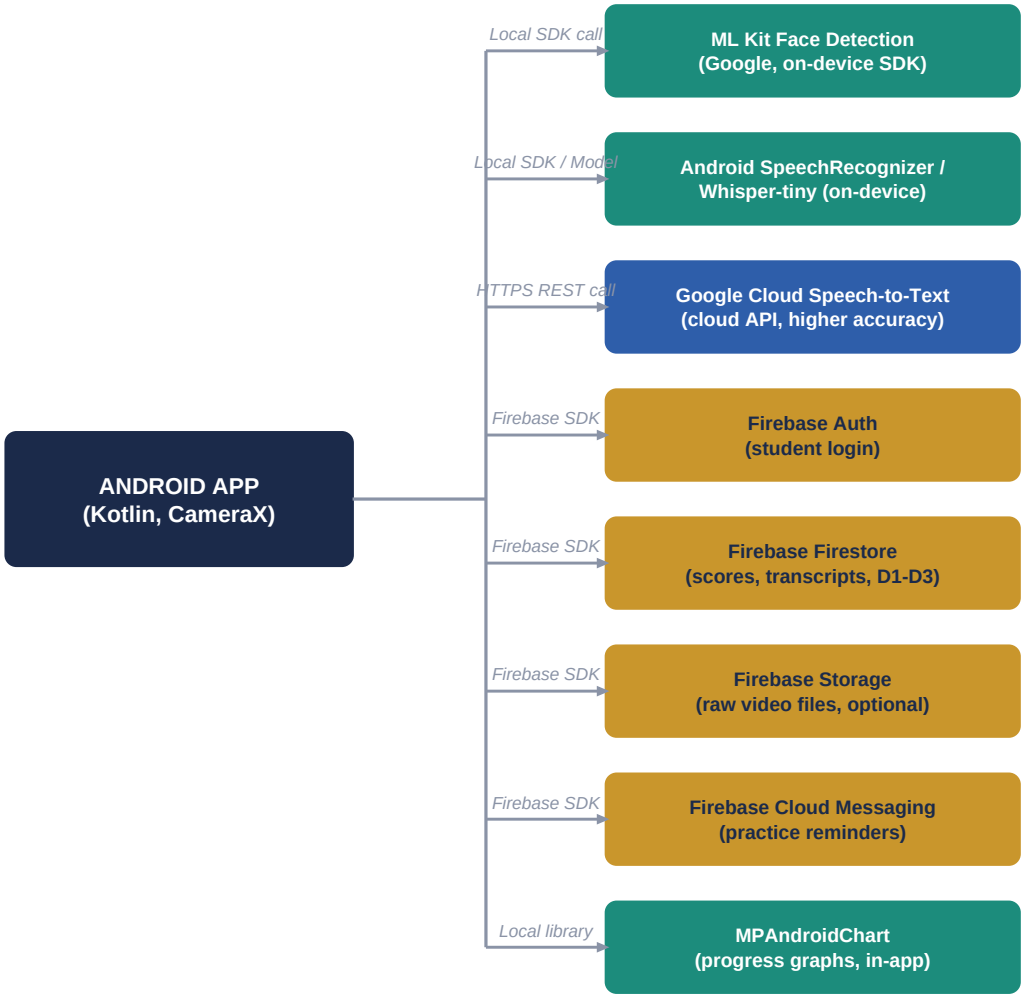

Fig 1.1 — How the Android app connects to every tool/platform

How to Read the Connections

- **Local SDK call:** code runs directly on the student's phone, instantly, no internet required.
- **HTTPS REST call:** the app sends a network request to a cloud server and waits for a response — needs internet, has small delay.
- **Firebase SDK:** a special case of cloud connection, but Google's Firebase library handles the networking for you — you just call simple functions.
- **Local library:** code that runs entirely inside the app with no network at all, e.g. drawing a chart from numbers you already have.

A Practical Note on Choice

You don't need both the on-device and cloud speech-to-text options — pick **one**. Start with Android's built-in **SpeechRecognizer** (it's free and needs zero setup) and only move to Google Cloud Speech-to-Text if you need higher accuracy for your final demo. This single decision saves the most setup time.

2. Tools & Platforms — One Per Concept

Concrete, current (2026) recommendations. "Recommended" is the one to start with for a semester project; "Alternative" is worth knowing but adds complexity or cost.

Concept	Recommended Tool	Alternative	Cost
Video/audio recording	CameraX (Android Jetpack)	Camera2 API (lower-level, harder)	Free
Speech-to-text	Android SpeechRecognizer (built-in, on-device)	Google Cloud Speech-to-Text API (cloud, more accurate)	Free / Pay-per-use after free tier
Filler word & pace scoring	Plain Kotlin string/regex logic	spaCy or basic NLP library (via server)	Free
Face & eye-contact detection	ML Kit Face Detection (Google, on-device)	MediaPipe Face Mesh (more detailed landmarks)	Free
Answer/STAR scoring	Keyword & pattern matching (Kotlin logic)	Hugging Face Inference API (semantic similarity model)	Free / Free tier + limits
Backend & database	Firebase Firestore	Supabase (Postgres-based)	Free tier, then pay-as-you-go
Login/auth	Firebase Authentication	Supabase Auth	Free tier
File storage (video)	Firebase Storage	Supabase Storage	Free tier (limited GB)
Push notifications	Firebase Cloud Messaging (FCM)	OneSignal	Free
Progress charts	MPAndroidChart	Vico (modern Compose charts)	Free, open-source
UI design/wireframes	Figma	Adobe XD	Free tier
Version control	GitHub	GitLab	Free
Testing	JUnit + Espresso	Robolectric	Free

Why These Specific Picks

- **Everything marked "Recommended" has a genuinely free tier** that's enough for a full semester demo — you will not hit billing surprises if you stay within Recommended columns.
- **Firebase is chosen over Supabase** mainly because Firebase has the deepest, most beginner-friendly Android documentation and official Google support inside Android Studio.
- **ML Kit over MediaPipe** for a first build — ML Kit is a simpler, higher-level API; move to MediaPipe only if your team wants finer-grained face landmark data later.

3. Getting Started — Setup Order

Follow this exact order in Android Studio. Doing it out of order is the most common reason teams get stuck early.

Step	What To Do	Where
1	Create a new Android project (Kotlin, Empty Activity)	Android Studio
2	Add CameraX dependencies, request camera + audio permissions	app/build.gradle + AndroidManifest.xml
3	Create a free Firebase project, connect it to your Android app (google-services.json)	firebase.google.com/console
4	Enable Firestore, Authentication, and Storage in the Firebase console	Firebase Console
5	Add SpeechRecognizer intent for speech-to-text (no extra key needed)	Android SDK (built-in)
6	Add ML Kit Face Detection dependency for eye-contact analysis	app/build.gradle
7	Add MPAndroidChart dependency for the progress screen	app/build.gradle (via JitPack)
8	Push initial project to a GitHub repo, add all teammates as collaborators	github.com

One Thing To Decide Before Day 1

Agree as a team on the Firebase project and GitHub repo *before* anyone starts coding modules separately — re-wiring a shared backend after each person has built against their own local setup is the single biggest time-waster in mini-projects like this.